

BTS CIEL option A
Cybersécurité, l'informatique et réseaux et l'électronique
E6 – PROJET INFORMATIQUE
Dossier de réalisation du projet.

Académies : Besançon-Dijon	Session : 2026
Lycée ou Centre de formation : Gustave Eiffel	
Ville : Dijon	
Nom du projet : Sechoir a houblon	
Projet industriel : OUI NON	

Equipe de développement.	
Professeur :	<i>Raoul Artola</i>
L'entreprise :	<i>Adrien Darphin</i>
Interlocuteur(s) de l'entreprise.	<i>Adrien Darphin (agriculteur)</i>
Etudiant 1 :	<i>Lénny Sauvage</i>
Etudiant 2 :	<i>Adam Cardoso</i>
Etudiant 3 :	<i>Ayoub Rabhi</i>

1. Présentation du projet

1.1 Problématique du projet

Le séchage du houblon est une étape critique qui doit être réalisée dans les 4 à 6 heures suivant la récolte, à une température maintenue entre 50 et 55°C, sans dépasser 60°C. Actuellement, ce processus est assuré manuellement, obligeant le propriétaire à rester en permanence sur place pour surveiller l'état du houblon. Cette contrainte est à la fois chronophage, fatigante et source d'erreurs humaines. Il devient donc nécessaire d'automatiser le contrôle du séchage afin de libérer l'exploitant tout en garantissant la qualité du produit et en optimisant la consommation énergétique.

1.2 Expression du besoin

Le système doit permettre à l'exploitant de piloter et surveiller à distance, depuis son smartphone, l'intégralité du cycle de séchage du houblon. Il doit automatiser la régulation de la température via le contrôle du brûleur à gaz et du ventilateur, mesurer en continu la température du séchoir, déclencher des alertes sonores et visuelles en cas de dépassement de seuil ou de fin de cycle, et enregistrer l'historique complet des données dans une base de données. L'interface web doit également permettre la saisie des paramètres de séchage, le suivi des variétés de houblon produites et l'export des données en fichier CSV sur clé USB.

1.3 Rôles par étudiant

Étudiant 1 — Interface utilisateur, RTC et base de données : il est responsable de la mise en œuvre de l'horloge temps réel (RTC), de l'installation du service web (Apache) et de la configuration de la base de données (MariaDB), de la conception de l'interface de saisie des paramètres utilisateur (variété, températures min/max, durée de séchage) ainsi que de la visualisation des données historiques sous forme de tableaux et graphiques. Il développe également la fonctionnalité d'export CSV.

Étudiant 2 — Acquisition des données et régulation thermique : il prend en charge le choix et la mise en œuvre des 6 capteurs de température, l'interfaçage électronique avec la Raspberry Pi, et le développement du programme de régulation tout ou rien du brûleur à gaz. Il gère également le déclenchement de l'alerte sonore (sirène) en cas de dépassement des 60°C, ainsi que l'enregistrement horodaté des mesures, commandes et alertes dans la base de données.

Étudiant 3 — Contrôle du séchage, alerte visuelle et connectivité WiFi : il est responsable de la configuration du point d'accès WiFi, du développement de la page web de contrôle du séchage (visualisation des températures, état du chauffage et de la ventilation, temps restant, alertes), de la mise en œuvre du voyant d'alerte visuelle de fin de cycle, et des fonctionnalités d'ajout de la masse de houblon produit. Il gère aussi l'enregistrement en base de données des événements de démarrage et d'arrêt des cycles.

1.4 Structure matérielle et logicielle du système

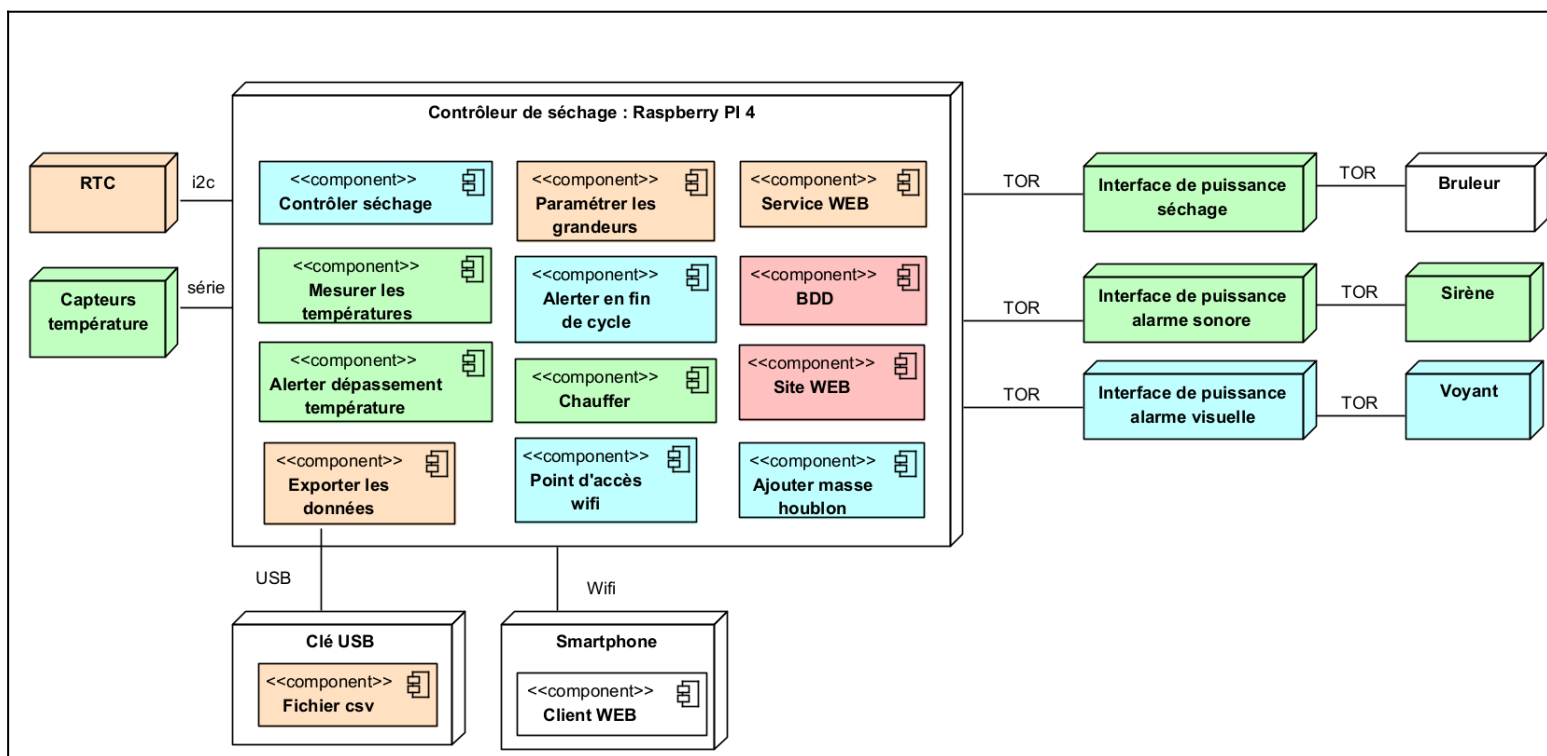
Matériel : le système repose sur une Raspberry Pi 5 comme contrôleur central, 6 capteurs de température étanches connectés en série, une RTC interne, des interfaces de puissance TOR commandant le brûleur à gaz, le ventilateur triphasé, la sirène et le voyant lumineux. L'armoire électrique est pré-câblée avec boutons Marche/Arrêt. Le smartphone sert de terminal de supervision via WiFi.

Logiciel : Raspberry Pi OS 64 bits, serveur web Apache 2, PHP 8.4, base de données MariaDB, développement en HTML5, CSS3, JavaScript, PHP et C. L'interface utilisateur est un site web responsive accessible depuis un navigateur mobile, sans installation côté client.

2. Analyse et spécifications

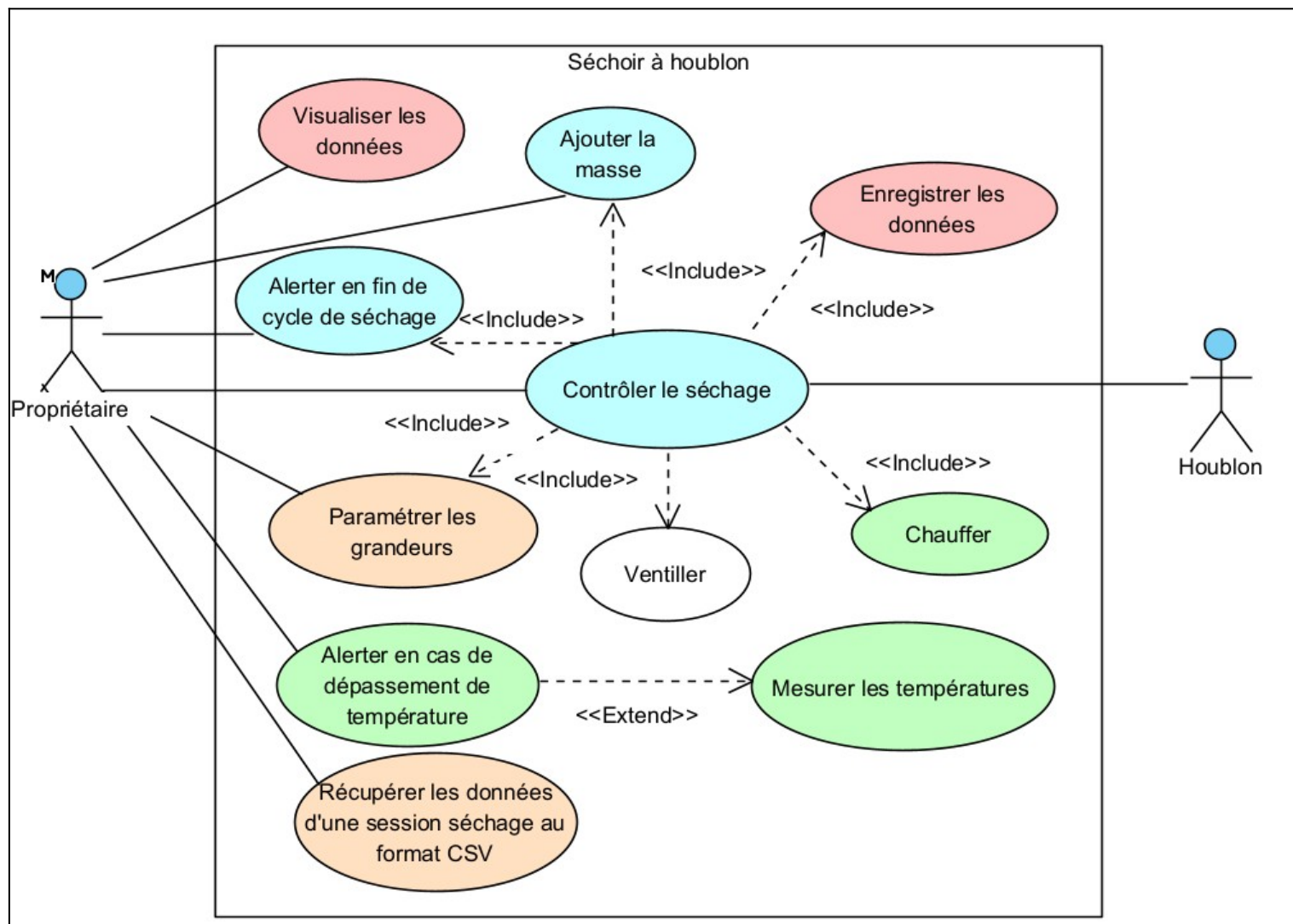
2.1 Diagramme de déploiement

Le système s'articule autour de la Raspberry Pi 5, qui constitue le nœud central de l'architecture. Côté matériel, elle reçoit les données des 6 capteurs de température via une liaison série, synchronise l'heure grâce à sa RTC interne, et pilote les trois interfaces de puissance TOR commandant respectivement le brûleur de séchage, la sirène sonore et le voyant lumineux. Côté logiciel, cette même Raspberry Pi héberge l'ensemble de la pile applicative : le service web Apache, le moteur PHP 8.4 et le serveur de base de données MariaDB. Le smartphone de l'utilisateur se connecte en WiFi et accède au site web pour superviser et contrôler le séchage en temps réel.



2.2 Diagramme de cas d'utilisation générale

Le système de séchage automatisé a été conçu pour répondre à un besoin concret : permettre au propriétaire de piloter et surveiller son séchoir à houblon à distance, sans nécessiter sa présence permanente sur les lieux. Le propriétaire peut ainsi configurer les paramètres de séchage, superviser en temps réel l'état du système, être alerté en cas d'anomalie ou de fin de cycle, et consulter l'historique des sessions passées.



2.3 Expression fonctionnelle du besoin

L'utilisateur principal du système est l'exploitant. Il interagit avec le système depuis son téléphone via un navigateur web connecté au point d'accès WiFi local du Raspberry Pi. Le système doit être simple d'utilisation, robuste et adapté à un environnement agricole.

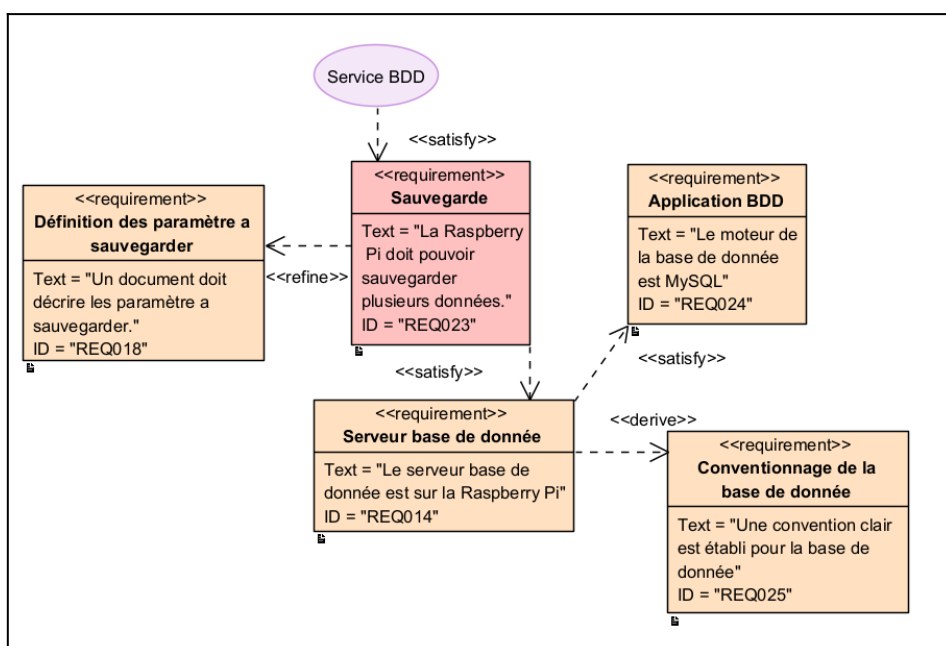
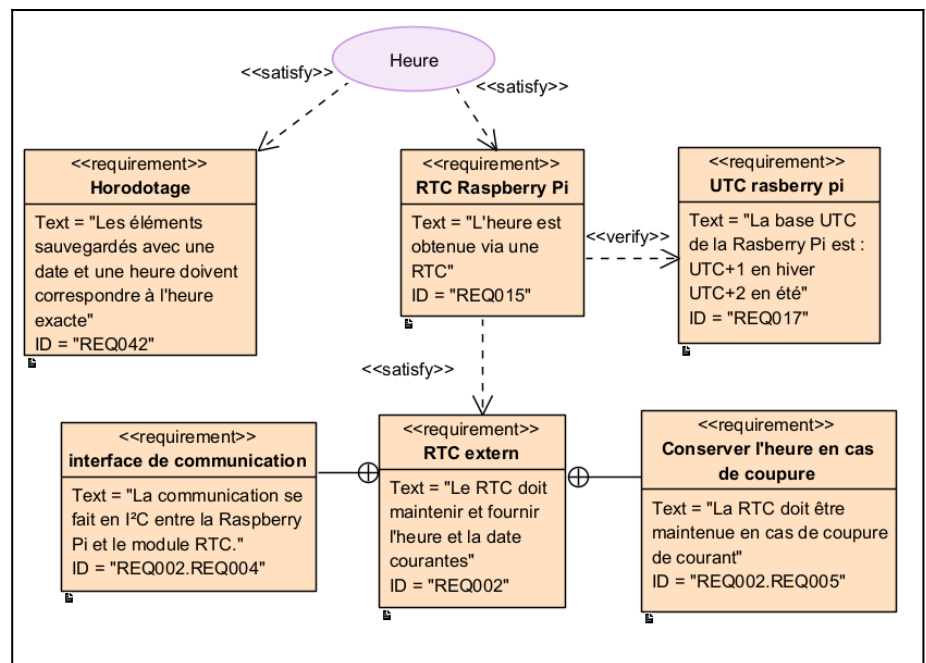
Fonction	Description attendue	Utilisateur concerné
Paramétrer un cycle	Saisie variété, températures min/max et durée de séchage.	Exploitant
Contrôler le séchage	Visualisation des 6 températures, moyenne, état chauffage/ventilation, temps restant.	Exploitant
Mesurer	Mesures toutes les minutes sur 6 capteurs.	Système

Chauffer	Commande tout ou rien du brûleur selon les seuils.	Système
Alerter	Sirène en cas de dépassement et voyant en fin de cycle.	Système
Enregistrer	Historique horodaté des mesures, actions et alertes.	Système
Exporter	Création d'un fichier CSV sur clé USB.	Exploitant

L'expression du besoin a été traduite en fonctions testables afin de faciliter la recette finale du projet.

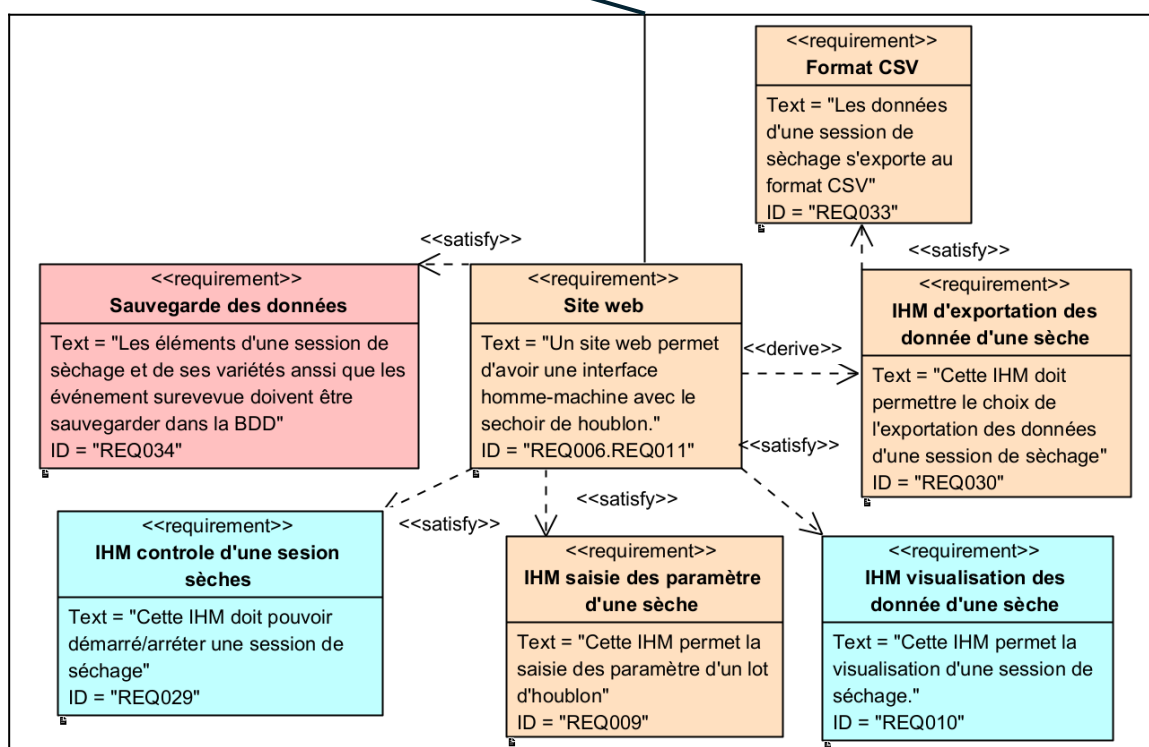
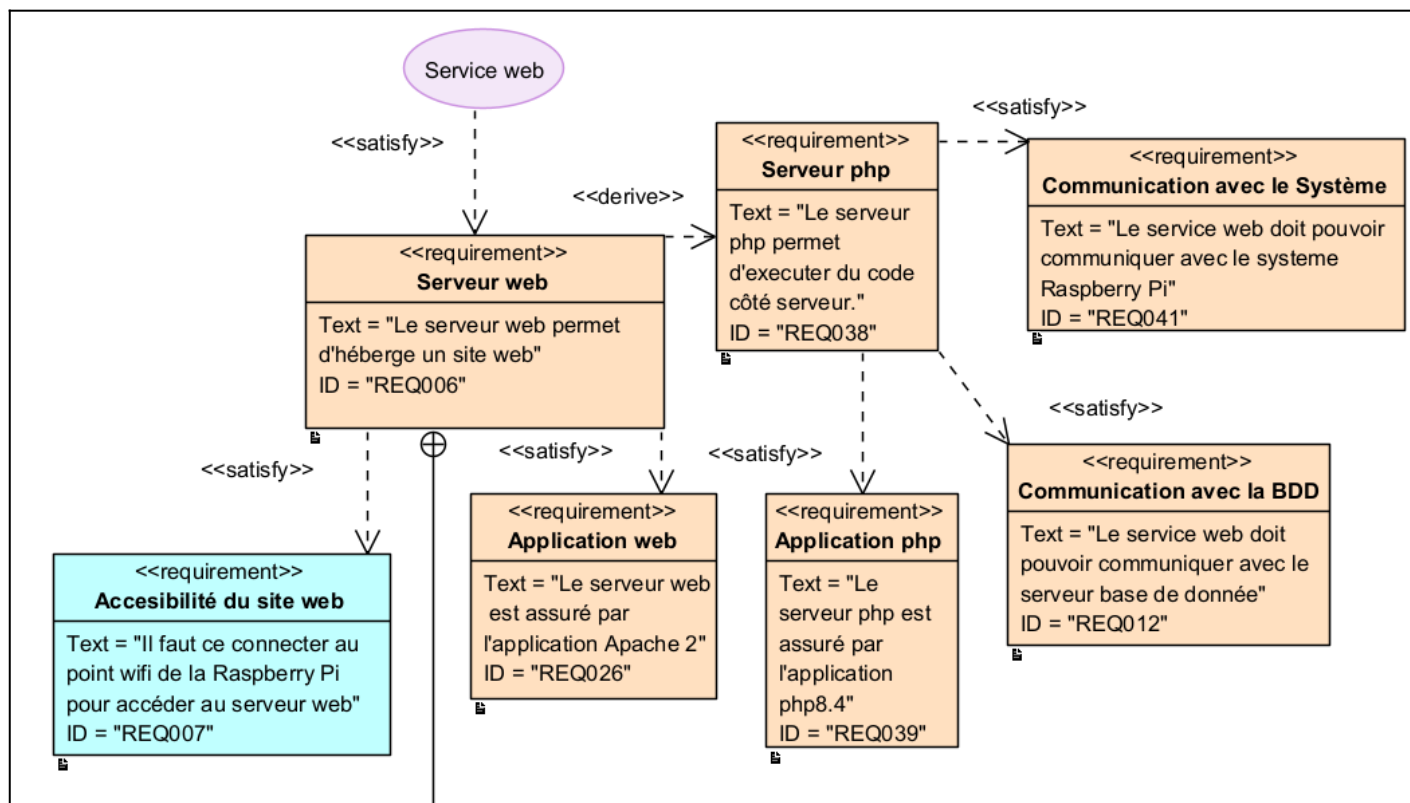
2.4 Diagramme d'exigences — RTC et base de données

Deux composants sont au cœur de la fiabilité du système : l'horloge temps réel (RTC) et la base de données. La RTC est indispensable pour garantir un horodatage précis de toutes les actions et mesures enregistrées tout au long du cycle de séchage. Elle maintient l'heure même en cas de coupure de courant. Le fuseau horaire est configuré en Europe/Paris. La base de données MariaDB est hébergée directement sur la Raspberry Pi et centralise les mesures de température, les événements de chauffage, les pauses et événements, et les informations liées aux variétés de houblon produites



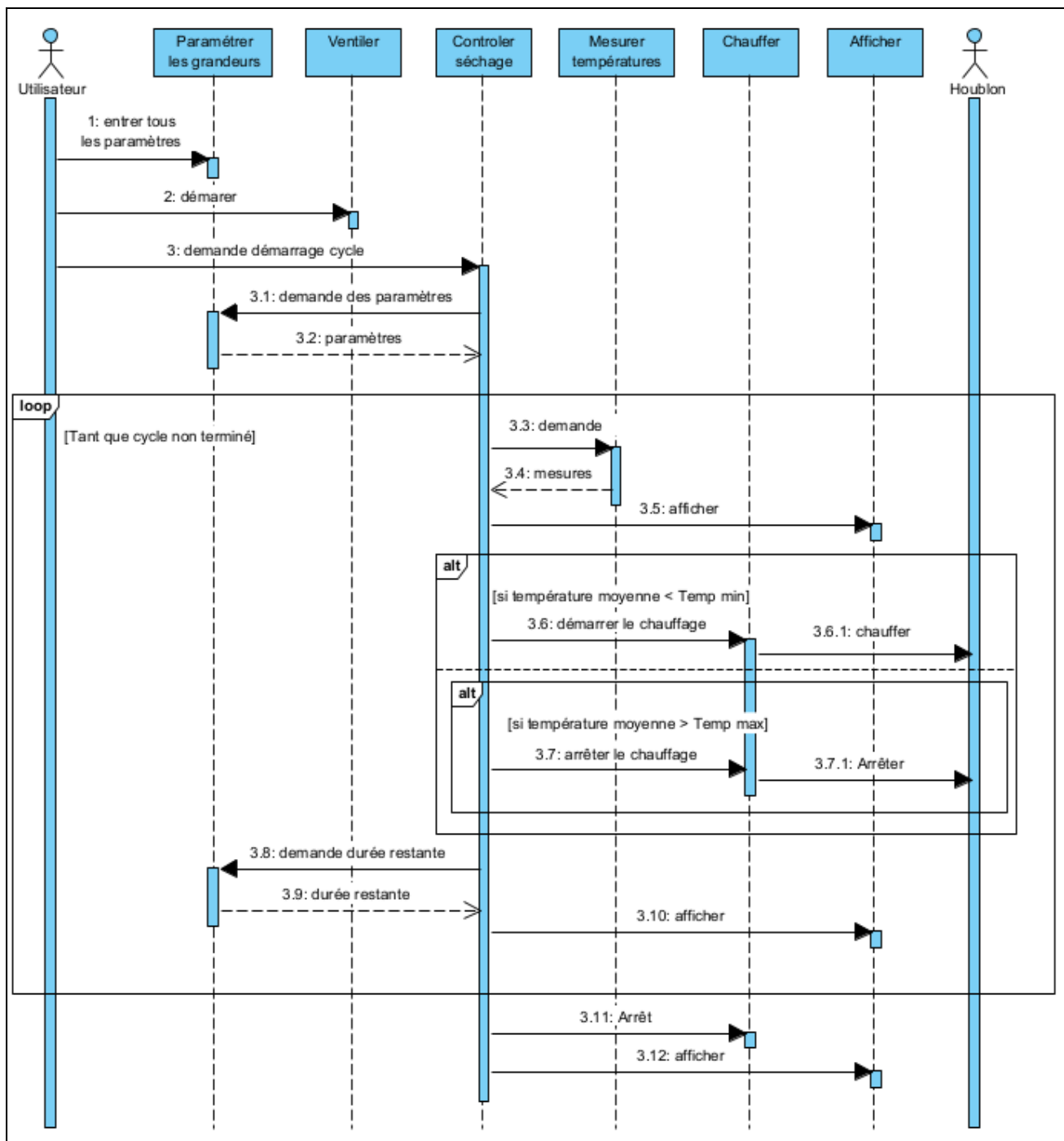
2.5 Diagramme d'exigences — Service web

L'interface utilisateur repose entièrement sur un service web hébergé sur la Raspberry Pi, accessible depuis le smartphone du propriétaire via le point d'accès WiFi dédié. Ce service est composé d'un serveur Apache 2 et d'un serveur PHP 8.4. Le service web communique avec la Raspberry Pi pour récupérer l'état du système en temps réel, et avec la base de données pour lire et écrire les informations liées aux cycles de séchage.



2.6 Diagramme de séquence

Le cycle de séchage débute par la saisie des paramètres par l'utilisateur : variété de houblon, températures minimale et maximale, et durée de séchage. Une fois ces paramètres validés, l'utilisateur démarre la ventilation depuis l'armoire électrique, ce qui lance officiellement le cycle. Le système entre alors dans une boucle de fonctionnement continu : les températures sont mesurées régulièrement et affichées sur l'interface web. Si la température moyenne descend en dessous du seuil minimal, le brûleur est automatiquement allumé. À l'inverse, si elle dépasse le seuil maximal, le brûleur est immédiatement coupé. En parallèle, le temps restant avant la fin du cycle est calculé et affiché en permanence. Lorsque la durée de séchage est écoulée, le système arrête le chauffage et en informe l'utilisateur via une alerte visuelle.



2.7 Évolutions et adaptations en cours de développement

Au cours du projet, plusieurs adaptations ont été nécessaires par rapport au cahier des charges initial. La principale est le remplacement de la Raspberry Pi 3 initialement prévue par une Raspberry Pi 5, fournie en cours de développement. Ce changement a eu des impacts directs : la Raspberry Pi 5 intègre nativement une RTC interne, supprimant le besoin d'un module I2C externe. La version du serveur PHP a également évolué vers PHP 8.4, et le serveur de base de données MySQL a été remplacé par MariaDB, compatible mais plus adapté à Raspberry Pi OS.

2.8 Tests réalisés sur le système

Des tests ont été menés de manière incrémentale, en validant chaque composant individuellement avant intégration dans le système complet. Chaque fonctionnalité développée a fait l'objet de tests unitaires documentés, couvrant les cas nominaux et les cas limites (dépassement de température, perte de connexion, données manquantes en base).

[Compléter ici avec le détail des tests réalisés et leurs résultats]

3. Méthodologie de prise en compte des contraintes du cahier des charges

La méthodologie adoptée tout au long du projet repose sur une lecture attentive du cahier des charges en amont de chaque développement, afin d'identifier les contraintes techniques et fonctionnelles à respecter avant toute implémentation. Les choix techniques ont été systématiquement justifiés et documentés.

Le travail a été organisé selon les rôles définis dans le cahier des charges, avec des points de synchronisation réguliers entre les trois étudiants afin d'assurer la cohérence de l'architecture globale, notamment au niveau de l'interface et/ou entre les scripts PHP de chaque étudiant et aussi pour la structure de la base de données.

4. Contenu du support de rendu

Le support de rendu CDROOM doit contenir : ce rapport, les codes sources (dernière version), ainsi que les manuels d'utilisation et d'installation du système. La hiérarchie recommandée est la suivante :

- Répertoire source/ : sources de tous les programmes.
- Répertoire bin/ : exécutables.
- Répertoire doc/ : documentations.
- Répertoire installation : fichiers d'installation de l'application.

Le projet étant coordonnée via git, un repo disponible en ligne sur github est accessible a l'url → <https://github.com/Sauvage89/sechoir-houblon>

La navigation au sein de ce repo est documenter dans le **README.md** qui ce trouve a la racine du projet.

Présentation du travail du candidat N°1 – Lénny Sauvage

Introduction

Dans le cadre du projet Séchoir à Houblon, ma contribution repose principalement sur trois axes : la mise en place de l'infrastructure système de la Raspberry Pi (serveur web Apache, base de données MariaDB, horloge temps réel), et le développement des deux interfaces utilisateur qui permettent à l'agriculteur de configurer les lots d'houblon et d'exporter l'historique de sa production.

Ce dossier présente, pour chacun de ces cinq sujets, le contexte et l'objectif, les choix techniques retenus, le périmètre exact de ma réalisation, ainsi que les difficultés rencontrées et les solutions apportées.

Tout test est vérifié par un cahier de recette.

Toute documentation technique se situe dans le dossier guide/ dans le CDROOM ou bien dans le dossier doc/guide/ du github du projet.

1. Configuration du service web (Apache 2 + PHP)

1.1 Contexte et objectif

L'agriculteur doit pouvoir interagir avec le système de contrôle depuis son smartphone, sans aucune installation logicielle particulière. Le choix d'une interface web s'est imposé comme solution naturelle : accessible depuis n'importe quel navigateur, elle ne nécessite aucun déploiement côté client et reste simple à maintenir.

Le serveur web est hébergé sur la Raspberry Pi 5 et constitue le point d'entrée unique pour la consultation des températures, le paramétrage du séchage, le pilotage du cycle (démarrage/arrêt), ainsi que la visualisation et l'export des données.

1.2 Choix techniques

L'environnement technique retenu repose sur les technologies imposées par le cahier des charges :

- Apache 2 : serveur HTTP, choisi pour sa robustesse, sa disponibilité sur Raspberry Pi OS et sa large documentation.
- PHP 8.4 : langage serveur permettant l'exécution de scripts et l'accès à la base de données.
- HTML5 / CSS3 / JavaScript / C : couche présentation côté navigateur, avec appels asynchrones (fetch) vers les scripts PHP.

1.3 Périmètre de ma réalisation

Ma contribution a porté sur l'installation et la configuration complète du serveur Apache 2 sur la Raspberry Pi, l'activation du module PHP, la définition des droits d'accès selon le principe de moindre privilège (utilisateur www-data), ainsi que la mise en place de la structure de fichiers du site web. La configuration du VirtualHost (fichier 000-default.conf) et la politique de droits sur les répertoires /var/www/html et /var/log/apache2 ont été définies et appliquées.

1.4 Installation et configuration

1.4.1 Installation d'Apache 2

L'installation du serveur web a été réalisée via le gestionnaire de paquets APT. Une vérification du bon fonctionnement a été effectuée en accédant à la page par défaut du serveur depuis un navigateur web sur le réseau local. Le service Apache a ensuite été configuré pour démarrer automatiquement au démarrage du système.

1.4.2 Installation de PHP 8.4

Le module PHP a été installé et intégré à Apache afin de permettre l'exécution de scripts côté serveur. Cette étape est essentielle car l'ensemble de la logique métier (traitement des données, interaction avec la base) repose sur PHP.

1.4.3 Organisation du site

Le site est déployé dans `/var/www/html`, avec la structure suivante :

- `api/` : scripts PHP (traitement, accès BDD)
- `site/` : pages HTML et PHP
- `css/` : feuilles de style (une par page)
- `js/` : scripts Javascript

Cette organisation permet une séparation claire entre front-end et back-end, facilitant la maintenance et le travail collaboratif.

1.4.4 Droits d'accès (principe de moindre privilège)

Le processus Apache s'exécute sous l'utilisateur système `www-data`, qui ne dispose que des droits strictement nécessaires :

- `/var/www/html` et ses sous-dossiers : droits 500 (`dr-x-----`) — parcours et lecture, pas d'écriture.
- Fichiers du site : droits 400 (`-r-----`) — lecture seule.
- `/var/log/apache2/access.log` et `error.log` : droits 600 (`-rw-----`) — lecture et écriture pour les journaux.
- `/etc/apache2/` : réservé à `root:root`, droits 700 récurifs.

Cette configuration garantit qu'une éventuelle faille dans le site web ne permettrait pas à un attaquant d'écrire ou de modifier des fichiers système.

1.4.5 Configuration du VirtualHost

Le serveur Apache est configuré via le fichier `000-default.conf` dans `/etc/apache2/sites-available/`. Ce fichier définit le port d'écoute (80), le répertoire racine (`/var/www/html`), et les chemins des journaux d'accès et d'erreurs. Le fichier actif dans `sites-enabled/` est un lien symbolique vers ce fichier disponible.

1.5 Fonctionnement des échanges front-end / back-end

Les pages du site ne se rechargent pas lors des interactions de l'utilisateur. La communication entre le navigateur et le serveur s'effectue de manière asynchrone via l'API fetch de JavaScript :

- ❑ L'utilisateur interagit avec l'interface (bouton, formulaire).
- ❑ Le JavaScript effectue une requête HTTP vers un script PHP de l'API.
- ❑ Le script PHP traite la demande, interroge ou met à jour la base de données, puis retourne une réponse au format JSON.

- ❓ Le JavaScript interprète la réponse JSON et met à jour dynamiquement l'affichage sans rechargement de page.

Cette approche permet une interface fluide et réactive, adaptée à une utilisation depuis un smartphone, sans nécessiter de framework JavaScript lourd.

2. Configuration du service base de données (MariaDB)

2.1 Contexte et objectif

L'enregistrement fiable des données est une exigence centrale du projet : températures relevées toutes les 30 minutes, événements système, cycles de séchage, lots et variétés de houblon, masses produites. Une base de données relationnelle MariaDB (compatible MySQL, imposé par le cahier des charges) a été retenue pour structurer, stocker et restituer ces informations.

L'objectif est de permettre à l'agriculteur de disposer d'un historique complet et cohérent, utilisable à la fois pour le suivi en temps réel via l'interface web et pour l'analyse rétrospective des campagnes de séchage.

2.2 Périmètre de ma réalisation

Mon travail a couvert la conception du schéma relationnel, la création des tables avec leurs contraintes d'intégrité (clés primaires, clés étrangères, valeurs nullables), ainsi que la définition des stratégies d'enregistrement pour chaque table. J'ai également rédigé la documentation technique de la base.

2.3 Architecture de la base

La base de données, utilisée par le compte www-data, est organisée autour de neuf tables couvrant l'ensemble du cycle de vie d'un lot de houblon :

Table	Description
pause	Enregistre les pauses horodatées survenant durant le fonctionnement du séchoir.
variete	Référentiel des variétés de houblon disponibles et désactivées.
masse	Enregistre la masse de houblon produite à l'issue du séchage.
etage	Données statiques représentant les étages physiques du séchoir.
etatSechoir	Table d'état courant du séchoir (variable globale persistante).
capteur	Référence les capteurs de température actifs et inactifs du système.
lot	Table centrale — suit un cycle complet de production pour une variété donnée.
temperature	Stocke les mesures de température relevées par les capteurs.

lotEtag

Table d'association reliant un lot à son étage courant, traçant ses changements d'étage.

2.4 Modèle Conceptuel de Données (MCD)

Un Modèle Conceptuel de Données est structuré autour de trois éléments : les **entités**, les **associations** et les **attributs**. Une entité représente un objet du système (par exemple un lot de houblon), ses attributs décrivent ses propriétés (variété, taux de remplissage, durée de séchage), et les associations modélisent les liens entre entités.

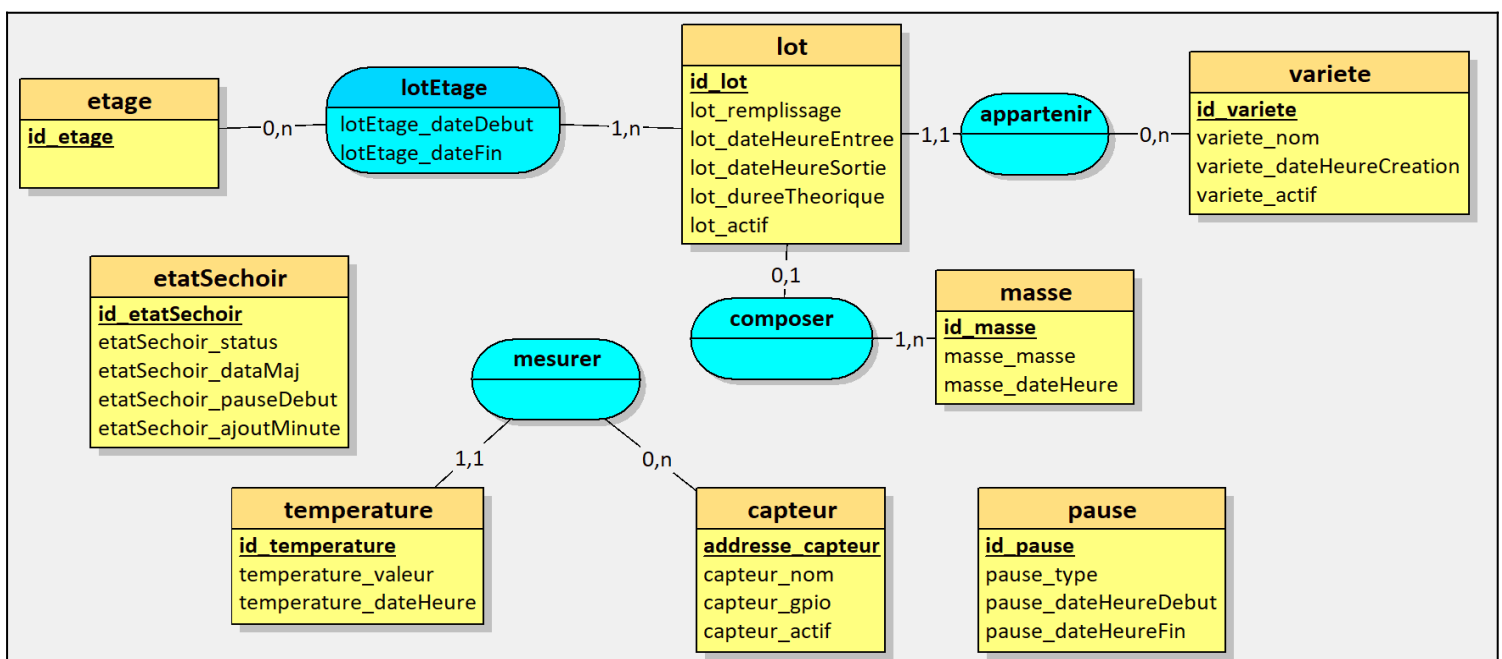
La lecture des associations se fait via les cardinalités, exprimées selon la notation suivante : (1,1) = 1 et 1 seul,

(0,1) = 0 ou 1,

(1,n) = entre 1 et n,

(0,n) = entre 0 et n.

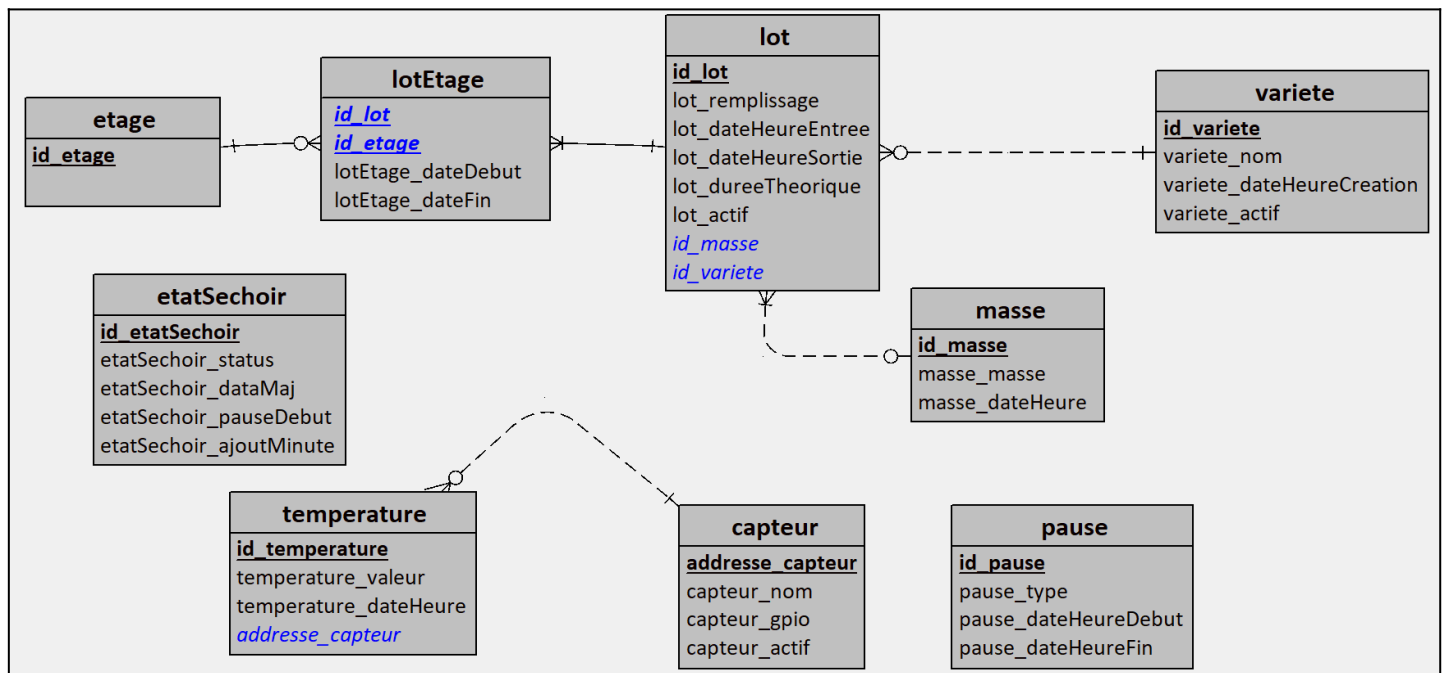
Par exemple, un lot appartient à une et une seule variété, tandis qu'une variété peut être associée à zéro à plusieurs lots.



2.5 Modèle Logique de Données (MLD)

Le Modèle Logique de Données est une traduction du MCD dans un format adapté à une implémentation en base de données relationnelle. Chaque entité devient une table, ses attributs deviennent des colonnes, et chaque table possède une clé primaire. Le passage du MCD au MLD dépend des cardinalités :

- (1,1) se traduit par une clé étrangère dans l'une des deux tables.
- (1,n) se traduit par une clé étrangère du côté « n ».
- (n,n) nécessite la création d'une table intermédiaire.



3. Configuration de l'horloge temps réel (RTC)

3.1 Contexte et objectif

Le séchoir est déployé dans une zone blanche, sans accès à Internet. La Raspberry Pi ne peut donc pas synchroniser son horloge via un serveur NTP. Or, l'horodatage correct de toutes les mesures et événements est une condition de fiabilité du système : des horodatages erronés rendraient l'historique inexploitable.

L'ajout d'une horloge temps réel (RTC) permet de conserver l'heure exacte même après une coupure d'alimentation, garantissant ainsi la cohérence des données à chaque redémarrage.

3.2 Solution retenue

Au cours du développement, une Raspberry Pi 5 a été mise à disposition. Contrairement à la Raspberry Pi 3 initialement prévue, la Pi 5 intègre nativement une RTC interne. Il suffit de brancher une pile sur le connecteur BAT de la carte pour alimenter cette RTC lorsque le système est hors tension, sans nécessiter de module externe ni de configuration I2C supplémentaire.

3.3 Périmètre de ma réalisation

J'ai réalisé une étude comparative des modules RTC externes courants (DS1307, DS3231, PCF8523) afin d'argumenter le choix technique initial. J'ai ensuite procédé à la configuration système : désactivation du faux RTC logiciel (fake-hwclock), réglage manuel de l'heure, écriture dans la RTC (hwclock -w), et vérification du bon fonctionnement après coupure d'alimentation.

3.4 Étude comparative des modules RTC externes

Avant la mise à disposition de la Raspberry Pi 5, une analyse des modules RTC disponibles a été conduite afin de sélectionner le composant le plus adapté : déploiement en zone blanche, environnement agricole soumis à des variations de température, et besoin de fiabilité sur une longue période sans maintenance.

Critère	DS1307	DS3231	PCF8523
Interface	I2C	I2C	I2C
Tension	5V	3,3V	3,3V
Précision	Moyenne	Haute (± 2 ppm)	Moyenne
Rétention pile	Variable	~6 ans	~6 ans
Coût moyen	~ 6,50 €	~ 7,90 €	~ 8,30 €
Compatible GPIO Raspberry Pi 3,3V	Non (adaptateur requis)	Oui	Oui

Conclusion : pour un usage critique en zone blanche où l'horodatage doit rester fiable sur toute une saison agricole, le DS3231 aurait été retenu. Sa tolérance aux variations thermiques et sa haute précision (± 2 ppm $\approx \pm 1$ minute par an) en font le module le plus adapté. La mise à disposition de la Raspberry Pi 5 a cependant rendu cette sélection caduque.

3.5 Analyse des risques liés à l'absence de RTC

Sans RTC fonctionnelle, à chaque coupure d'alimentation la Raspberry Pi redémarre avec une heure incorrecte. Les conséquences directes sont :

- Les mesures de température en base seraient horodatées incorrectement, rendant tout graphique d'évolution thermique inexploitable.
- Les événements en base (démarrage de cycle, alerte, arrêt) perdraient leur valeur chronologique, rendant impossible tout audit rétrospectif.

3.6 Procédure de configuration — Raspberry Pi 5 (RTC interne)

Étape 1 — Connexion de la pile

Brancher une pile bouton compatible sur le connecteur BAT de la Raspberry Pi 5. Cette pile alimentera exclusivement le circuit RTC lorsque la carte sera hors tension.

Étape 2 — Réglage initial de l'heure

Lors de la première mise en service, l'heure doit être renseignée manuellement puisque le système est hors réseau :

```
sudo timedatectl set-timezone Europe/Paris
sudo timedatectl set-time "YYYY-MM-DD HH:MM:SS"
```

Étape 3 — Vérification du fonctionnement

Après un redémarrage, la commande suivante doit afficher l'heure correcte :

```
sudo timedatectl status
```

Étape 4 — Vérification finale après coupure

Pour valider le comportement lors d'une vraie coupure d'alimentation : couper l'alimentation, patienter quelques minutes, remettre sous tension, puis exécuter `date` et `sudo timedatectl status` pour confirmer que l'heure est restée correcte.

3.7 Bilan

La RTC constitue un prérequis technique indispensable au bon fonctionnement du système dans un contexte hors réseau. La Raspberry Pi 5, en intégrant nativement ce composant, simplifie considérablement le câblage et la configuration par rapport à la solution initialement envisagée avec un module I2C externe. La procédure de configuration a été testée et validée : après coupure d'alimentation et redémarrage, l'heure système est correctement restituée depuis la RTC interne.

4. Développement de l'IHM de configuration du houblon

4.1 Contexte et objectif

Avant de démarrer un cycle de séchage, l'agriculteur doit renseigner un ensemble de paramètres : la variété de houblon chargée à chaque étage, le taux de remplissage, la température cible (min/max) et la durée prévue du cycle pour le lot qu'il est entrain de renseigner. Cette saisie doit être possible depuis un smartphone, sans compétence informatique particulière. Ces paramètres sont ensuite sauvegardés en base de données dès le lancement du cycle.

4.2 Fonctionnalités développées

- Sélection de la variété de houblon par étage (liste déroulante alimentée depuis la table `variete`).
- Saisie du taux de remplissage de l'étage (boutons – et +, par pas de 10 %, entre 0 et 100 %).
- Saisie de la durée du cycle au format hh:mm avec boutons d'ajustement par pas de 10 minutes.
- Affichage récapitulatif de tous les paramètres.
- Enregistrement en base de données via un script PHP.
- Adaptation de l'interface selon le contexte (création ou modification d'un lot existant).

4.3 Périmètre de ma réalisation

J'ai conçu et développé la page HTML/CSS/JavaScript correspondante, ainsi que le script PHP côté serveur chargé de valider les données saisies et de les persister en base. L'interface a été pensée pour une utilisation mobile (responsive) via la librairie « bootstrap », avec un retour visuel immédiat à l'utilisateur après chaque action .

4.4 Architecture technique de l'IHM

4.4.1 Structure des fichiers

L'interface de configuration se présente sous la forme d'un overlay (fenêtre modale) qui s'ouvre au clic sur le bouton de chaque étage depuis la page principale de gestion du séchoir. Ce choix technique évite une navigation entre plusieurs pages et permet à l'agriculteur d'agir rapidement, sans perdre de vue l'état général du séchoir.

Le code est découpé en deux fichiers PHP distincts : `gestion_sechoir.php` qui affiche la vue principale des quatre étages, et `gestion_sechoir_overlay.php` qui contient la fenêtre modale de configuration.

4.4.2 Architecture JavaScript modulaire

Côté JavaScript, j'ai adopté une architecture modulaire en séparant les responsabilités en plusieurs fichiers distincts, chargés de manière différée (`defer`) :

- `GestionAPI.js` : gestion des appels vers l'API REST de la Raspberry Pi (`get_status.php`, création/suppression de lot).
- `GestionUI.js` : manipulation du DOM, affichage et masquage de l'overlay, mise à jour des champs.
- `GestionUTILS.js` : fonctions utilitaires (validation du format hh:mm, calcul du pourcentage de remplissage).
- `GestionSTATE.js` : gestion de l'état courant de l'overlay (étage sélectionné, lot existant ou nouveau).
- `index.js` : point d'entrée, initialisation et liaison des événements.

Ce découpage facilite la maintenance et la lecture du code.

4.5 Comportement de l'overlay selon le contexte

L'overlay adapte son comportement selon qu'un lot existe déjà sur l'étage concerné ou non. Lors de l'ouverture, l'état courant de l'étage est récupéré depuis l'API : si un lot est déjà enregistré, les champs sont pré-remplis avec ses données et le bouton principal affiche « Modifier le lot » ; sinon, le formulaire est vide et le bouton affiche « Créer un lot ».

The screenshot shows a modal window titled "Étage 4" with a close button (X) in the top right corner. The subtitle is "Configuration d'un lot de houblon". A red message states "Cette étage ne contient pas de lot". Below this, there are several input fields and buttons:

- "Variété de houblon" with a dropdown menu showing "-- Sélectionner une variété --".
- "Remplissage" with a numeric input set to "50" and percentage symbols, flanked by minus and plus buttons.
- "Temps de séchage voulue" with a time input set to "00:00" and minus/plus buttons.
- Two buttons at the bottom: "Créer un lot" and "Supprimer ce lot".
- A "Quitter" button at the very bottom of the modal.

In the bottom right corner of the background interface, there is a temperature display: "CAPTEUR 6 18.3 °C 09:13".

4.6 Communication avec l'API

La sauvegarde des paramètres s'effectue via une requête fetch vers un endpoint PHP côté serveur. Les données sont envoyées au format JSON et le script PHP les valide puis les insère ou les met à jour en base de données.

Script de connexion à la base de donnée

Cette fonction retourne un objet « PDO » qui est une interface de communication avec la base de donnée.

Le « ? » permet de dire que cette objet peut être NULL si une erreur est survenue lors de la tentative de connexion.

Appeler cette fonction permet de récupérer un moyen de communiquer avec la base de données.

```

2  function db_connect(): ?PDO
3  {
4      $host  = "localhost";
5      $dbname = "base_sechoir";
6      $user   = "dbsechoir";
7      $pass   = "password";
8      $pdo    = null;
9
10     try
11     {
12         $pdo = new PDO(
13             "mysql:host=$host;dbname=$dbname;charset=utf8",
14             $user,
15             $pass,
16             [
17                 PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
18                 PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
19             ]
20         );
21
22         return ($pdo);
23     }
24     catch (PDOException $e)
25     {
26         die("Erreur connexion BDD : " . $e->getMessage());
27         return (NULL);
28     }
29 }

```

Le bloc **try...catch** : Il sert à capturer les erreurs de connexion (exceptions). Cela est crucial pour la sécurité, car cela empêche PHP d'afficher par erreur vos identifiants (nom d'utilisateur et mot de passe) sur l'écran de l'utilisateur en cas de panne du serveur.

L'option **PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION** : Indispensable pour le développement, elle force PHP à générer une erreur fatale (exception) dès qu'une requête SQL est mal écrite, rendant le débogage beaucoup plus rapide.

L'option **PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC** : Cette configuration définit le mode de récupération par défaut. Elle permet de recevoir les données sous forme de tableaux associatifs (clé => valeur) automatiquement, sans avoir à le préciser à chaque requête.

L'encodage **charset=utf8** : Il garantit que les caractères spéciaux et les accents soient correctement lus et enregistrés dans la base de données dès l'établissement de la connexion.

Pour finir, l'instanciation **new PDO(...)** : C'est l'étape précise où PHP tente de se connecter physiquement à la base de données.

```
31 function db_query(PDO $pdo, string $sql, array $params = []): PDOStatement
32 {
33     $stmt = $pdo->prepare($sql);
34     $stmt->execute($params);
35     return ($stmt);
36 }
37 ?>
38
```

Script d'envoi de requête à une base de donnée

Cette fonction permet d'envoyer une requête à une base de donnée.

Elle retourne un objet « PDOStatement » qui est le résultat de la requête exécutée et/ou contient les erreurs survenues lors de la requête.

Le paramètre \$params = [] (Argument optionnel) : C'est une sécurité très importante. Le fait d'utiliser un tableau vide par défaut permet deux choses :

- **Si tu ne mets rien** : La fonction comprend qu'il n'y a pas de variables à filtrer (ex: "Afficher tout").
- **Si tu mets des valeurs** : Elle les utilise pour remplir les "trous" de ta requête SQL en les nettoyant, ce qui protège ton site contre les **injections SQL** (piratage).

\$pdo->prepare(\$query_sql) (La préparation) : Avant d'envoyer les données, la fonction envoie le "plan" de la requête au serveur. La base de données analyse la structure de la commande sans l'exécuter, ce qui améliore les performances si on répète la même action.

\$stmt->execute(\$params) (L'exécution) : PHP envoie les données réelles pour compléter la requête. Si une erreur de syntaxe SQL est présente, c'est à ce moment-là que l'exception (l'erreur fatale) sera déclenchée grâce aux réglages faits dans db_connect.

4.7 Mise à jour automatique de l'affichage

La page de gestion du séchoir interroge régulièrement l'API `get_status.php` pour récupérer l'état en temps réel de chaque étage et des six capteurs de température. La fonction `rafraichirStatus()` est appelée au chargement de la page et relancée périodiquement. Les données reçues alimentent directement les éléments du DOM via les fonctions `updateEtatCycle()`, `updateEtage()` et `updateTemperature()`. La température moyenne des six capteurs est calculée côté client et affichée dans un bloc dédié.

4.8 Difficultés rencontrées et solutions apportées

La principale difficulté a été la gestion de la cohérence entre l'état affiché et l'état réel de la base de données, notamment lors d'actions rapides successives (création puis suppression immédiate d'un lot). J'ai résolu ce point en systématisant le rechargement des données depuis l'API après chaque action, plutôt que de modifier l'affichage localement.

Un autre point d'attention a été la validation du format de saisie de la durée (hh:mm). La validation est effectuée en JavaScript avant l'envoi, avec un retour visuel immédiat sous le champ concerné, sans bloquer l'utilisateur.

Et la dernière difficulté était qu'on était 2 étudiants à devoir développer une même page, ce qui a nécessité une bonne coordination d'équipe pour l'avancement du projet.

5. Développement de l'IHM d'export de données

5.1 Contexte et objectif

À l'issue d'une saison de séchage, l'agriculteur doit pouvoir accéder à l'historique complet de ses productions : variétés séchées, dates et durées des cycles, températures relevées, masses finales obtenues. Le cahier des charges prévoit la visualisation de ces données sous forme de tableaux et graphiques, ainsi que leur export au format CSV.

5.2 Fonctionnalités développées

- Affichage des données de production historiques (variété, dates, masse en kg) sous forme de tableau.
- Visualisation graphique de l'évolution des températures par cycle de séchage.
- Filtrage des données par variété ou par numéro de lot.
- Export des données sélectionnées au format CSV.

5.3 Périmètre de ma réalisation

J'ai développé la page d'export dans son intégralité : interface HTML/CSS/JavaScript pour l'affichage et le filtrage, scripts PHP pour l'interrogation de la base de données et la génération du fichier CSV.

5.4 Architecture technique de la page

5.4.1 Parcours utilisateur en trois étapes

La page d'export est structurée selon un parcours utilisateur en trois étapes successives, matérialisées par trois sections qui s'affichent progressivement : le choix du type d'export, puis les filtres, puis les résultats. Cette progression guidée évite de surcharger l'écran et reste lisible sur smartphone.

Deux modes d'export ont été prévus : l'export par lot, qui permet d'obtenir le détail complet d'un cycle de séchage précis par lot (pleinement fonctionnel), et l'export par production, destiné à regrouper tous les lots d'une saison (en développement).

5.4.2 Chargement dynamique des données de filtrage

Dès le chargement de la page, la liste des variétés de houblon est récupérée automatiquement depuis la base de données via un appel à l'API `query_get_variete.php`. Les options du filtre sont générées dynamiquement par rapport aux type d'export.

5.4.3 Filtrage et prévisualisation des résultats

Lorsque l'utilisateur clique sur « Rechercher », la fonction `loadTable()` collecte les valeurs des champs de filtre actifs et les envoie au script PHP `query_get_lots_preview.php` via une requête POST. Les filtres sont collectés par une fonction générique `getFiltersRender()` qui parcourt un dictionnaire de correspondances entre les noms de champs logiques et les identifiants HTML, rendant le code extensible sans modification de la logique de collecte.

Les résultats sont affichés sous deux formes simultanées générées par la même fonction `renderTable()` : un tableau classique pour les écrans larges, et un affichage en cartes empilées pour mobile. Les statuts des lots (Terminé, En cours, Erreur) sont mis en valeur par des badges colorés définis dans le dictionnaire `BADGE`.

5.4.4 Génération et téléchargement du fichier CSV

L'export d'un lot individuel est déclenché par le bouton CSV de chaque ligne du tableau. La fonction `exportRow()` envoie les filtres actifs ainsi que le numéro du lot ciblé à l'API `query_export_csv.php`, qui retourne un objet JSON structuré contenant quatre blocs de données : les informations du lot, le passage par étage (avec dates de début et de fin de séjour), les relevés de température, et les événements (pauses, arrêts).

La construction du fichier CSV est réalisée entièrement côté JavaScript, sans rechargement de page. Chaque bloc est précédé d'un en-tête de section (=== TEMPERATURES ===, etc.) pour faciliter la lecture dans un tableur. Les blocs températures et événements sont optionnels selon les cases cochées dans les filtres. Le fichier générée est proposé au téléchargement.

	A	B	C	D	E	F	G
1	=== LOT ===						
2	id_lot	lot_replissage	lot_dateHeureEntree	lot_dateHeureSortie	lot_dureeTheorique	lot_actif	variete_nom
3	39	80	2026-04-29 14:14:32	2026-04-29 14:14:39	30	0	Cascade
4							
5	=== ETAGES ===						
6	id_lot	id_etage	lotEtage_dateDebut	lotEtage_dateFin	duree_minute		
7	39	1	2026-04-29 14:14:38	2026-04-29 14:14:39	0		
8	39	2	2026-04-29 14:14:36	2026-04-29 14:14:38	0		
9	39	3	2026-04-29 14:14:32	2026-04-29 14:14:36	0		
10							
11	=== TEMPERATURES ===						
12	id_lot	id_temperature	temperature_valeur	temperature_dateHeure	adresse_capteur		
13	39	37	18.2	2026-04-29 14:14:32	CAP1		
14	39	38	18.3	2026-04-29 14:14:33	CAP1		
15	39	39	18.4	2026-04-29 14:14:34	CAP1		
16	39	40	18.6	2026-04-29 14:14:35	CAP1		
17	39	41	18.7	2026-04-29 14:14:36	CAP1		
18	39	42	18.8	2026-04-29 14:14:37	CAP1		
19	39	43	18.9	2026-04-29 14:14:38	CAP1		
20	39	44	19.0	2026-04-29 14:14:39	CAP1		
21							
22	=== EVENEMENTS ===						
23	id_pause	pause_type	pause_dateHeureDebut	pause_dateHeureFin			
24	1	pause_technique	2026-04-29 14:14:35	2026-04-29 14:14:36			
25	2	non	2026-04-29 14:14:35	2026-04-29 14:14:36			
26	3	non	2026-04-29 14:14:38	2026-04-29 14:14:40			
27	4	pause_technique	2026-04-29 14:14:38	2026-04-29 14:14:40			
28							
29							

5.5 Difficultés rencontrées et solutions apportées

La principale difficulté a été la structuration du fichier CSV de sortie pour qu'il soit à la fois exploitable dans un tableur et lisible en texte brut. L'ajout de séparateurs de sections textuels (=== LOT ===, === ETAGES ===, etc.) est un compromis qui facilite la compréhension sans perturber une importation automatisée, les en-têtes de colonnes étant systématiquement présents après chaque séparateur.

Une autre difficulté a été la gestion de l'état courant de l'interface entre la prévisualisation du tableau et le déclenchement de l'export. Deux fonctions distinctes ont été créées à cet effet : `getFiltersRender()` ignore les cases à cocher pour la prévisualisation, tandis que `getFilters()` les inclut pour la génération du CSV.

Conclusion

Ce dossier présente l'ensemble des réalisations dont j'ai eu la responsabilité dans le cadre du projet Séchoir à Houblon. Les cinq sujets traités — service web, base de données, RTC, IHM de configuration et IHM d'export — forment un ensemble cohérent couvrant l'intégralité de la chaîne : de la configuration de l'infrastructure système jusqu'à l'interface utilisateur finale.

Les principales difficultés rencontrées ont été la gestion de la cohérence des données entre l'interface et la base, la structuration d'un fichier CSV lisible par différents outils, et l'adaptation au changement de matériel (Raspberry Pi 3 vers Pi 5) en cours de projet. Ces défis ont été surmontés grâce à une approche incrémentale et à une documentation rigoureuse de chaque étape.

Le CDROM.

Le CDROM doit contenir :

- ☐ ce rapport mentionné ci-dessus ;
- ☐ les codes sources ;
- ☐ les manuels d'utilisation et d'installation du système.

Si le projet est composé de N sous-systèmes, la hiérarchie du CDROM pourra être la suivante :

- ☐ Répertoire **source** : contiendra les sources de vos différents programmes. **Dernière version.**
- ☐ Répertoire **bin** : contiendra les exécutables de vos différents programmes et les fichiers Makefiles s'ils existent. **Dernières versions.**
- ☐ Répertoire **doc** : contiendra toutes les documentations au format pdf des constructeurs qui ont été nécessaires pour pouvoir réaliser l'application.
- ☐ Répertoire **textes** : contiendra les fichiers textes du rapport, des notices d'installation et d'utilisation et du carnet de bord. **Dernières versions.**
- ☐ Répertoire **installation** : contiendra le ou les fichiers d'installation de l'application. Celle-ci devra pouvoir se faire en automatique (les exécutables, les DLL si nécessaires, etc.).

Le CD devra avoir une étiquette contenant :

- ☐ Le nom du projet.
- ☐ Les noms des étudiants.
- ☐ L'académie et le nom du lycée.
- ☐ L'année de la session.